Agentic PV systems: solar panels that investigate themselves

Wei He, King's College London (wei.4.he@kcl.ac.uk) · 30 October 2026 · Code and notebooks: github.com/hewei8622/agent_pv

A solar installation today can tell you that its output has dropped. It cannot tell you why, what it checked, or how sure it is. This programme builds software that can: an **agent** that lives beside the panel, senses it in several ways, reasons about what could be wrong, chooses its own measurements within a budget and declared limits, and talks to the people responsible for it. This note, a companion to the programme brief, explains what we do, how, what exists and what comes next. Each section ends with why that stage matters.

1 What the project is about

Monitoring in a modern solar plant is a smart system: it watches one number, usually power, and raises an alarm when a fixed rule is broken. The alarm carries no cause. An engineer is sent, often weeks later, to find out whether the panel was shaded, dirty, overheated, wired badly, or whether a cloud simply passed. Anything not on the designer's fault list stays silent (Figure 1, top).

An agentic PV system replaces the fixed rule with a loop that behaves more like a careful technician (Figure 1, bottom). In plain terms it has five properties that a smart system does not:

- it **keeps a belief** about its own condition, with uncertainty, rather than a yes/no flag;
- it **chooses its evidence**: the next picture or electrical test is picked by how much it would change the answer and what it costs, and it can decide to stop;
- it **acts within declared limits and checks**: it predicts what a test should do, does it, and compares;
- it **copes with the unforeseen**: new goals, new conditions, causes that were not on any list;
- it **stays in conversation with the user**: it reports what it believes and how sure it is, asks before a costly step, takes a new goal given in words, and can end with "unresolved, and this test would settle it" or "please send a person".

The programme asks two families of questions. **Case A** looks inward: "my output is down; why, and how sure am I?", including causes the agent was never told about. **Case B** looks outward: "given the weather, the tariff and who is in the building, what should I do now?", with a battery and a demand attached. The agent core is the same in both; only the domain knowledge changes. The work is organised as a **minimal goal**, reachable within one project module, and **stretch goals** that build on it (Section 5). The minimal goal is a simulator, an agent proven in it, and a bench prototype that shows the same agent running live. Beyond finding causes, a stretch goal is a **predictor**: a model of the panel and its surroundings that says what will happen next and what acting or waiting will cost. Diagnosis and prediction feed each other. Every investigation is a small experiment with a known outcome that teaches the predictor, and anything the predictor cannot explain is a lead for the next investigation. The research question behind all of it: *what does a domain-specialised agent add over a competent smart system, and over a general language-model agent given the same tools?*

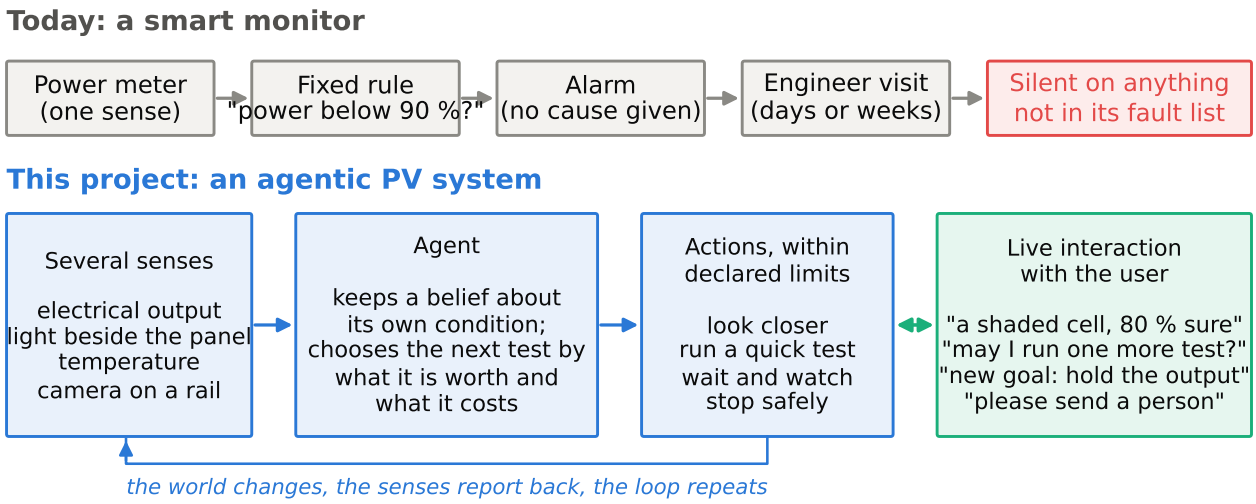


Figure 1: From alarm to conversation. A smart monitor (top) turns one sense into an alarm and leaves the diagnosis to a person. The agentic system (bottom) combines several senses, keeps a belief, chooses actions within limits, and keeps the user in a two-way exchange: it reports, asks, and takes new goals.

Why it matters. Faults not on the list stay silent for months, and an engineer visit often costs more than the energy it recovers. A belief with a stated confidence can be acted on; an alarm cannot. The core is device-independent, so the same work pays off for batteries and buildings.

2 How we approach it

Everything the agent does is one loop (Figure 2, left). It **senses**, through readings that arrive with real rates and delays. It **believes**: which causes are still possible and how likely. It **decides** which measurement is worth its cost. It **acts**: moves the camera, runs a short electrical test, waits, or stops safely. It **verifies** that the world responded as predicted. It **finishes** by reporting to the user, who can reply. Its budget in time, energy, images and test points is counted.

We build and prove this **digitally first** (Figure 2, right). A simulator with hidden causes lets us run hundreds of cases where the true cause is known to the scorer but not to the agent. A fixed rule, a competent classical planner and the agent all face the same cases, and one evaluator scores them, rewarding a correct cause, penalising a confident wrong claim, and giving credit for "unresolved" when the agent names the test that would settle it. Only then does the same code move to a physical bench rig, where it replays what the digital runs found.

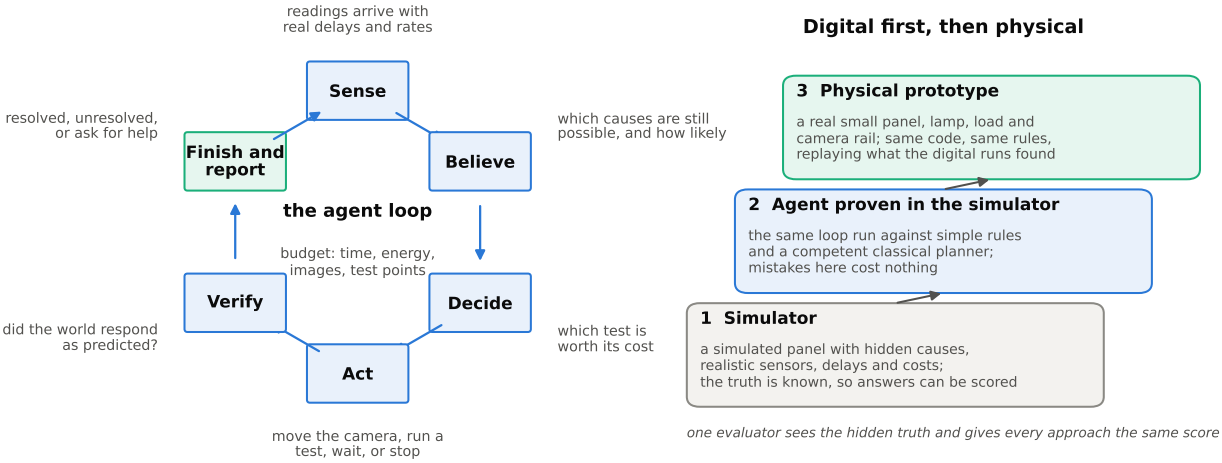


Figure 2: The approach. Left: the agent loop, with its budget in the middle. Right: three stages, each one proved against the last, with one evaluator that sees the hidden truth.

How the agent is built. We plan to run the agent inside **OpenCode**, an open-source agent framework that already handles conversation, memory and tool use (Figure 3). The agent has four parts: a **model** that reasons, **instructions and PV knowledge** that tell it its role and the physics it may rely on, a **session memory** that holds its belief and the evidence so far, and **helpers** for narrow jobs such as reading a picture or running a test. It touches the world only through **tools**, and every tool sits behind a permission: allowed, allowed after asking the user, or denied. The same tool names connect to the simulator, to the bench rig, or to outside systems such as a weather forecast or a tariff, so the agent does not need to know which world is behind them.

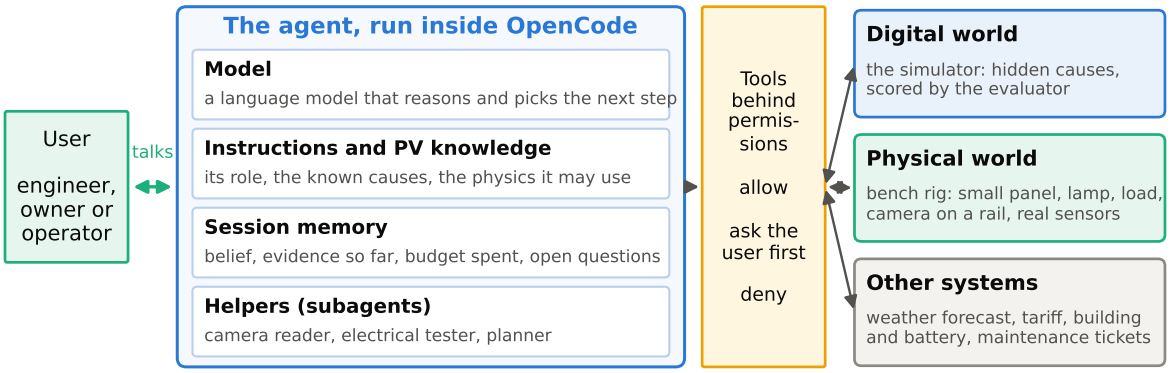


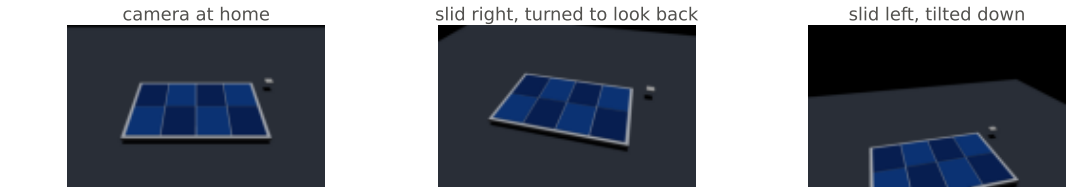
Figure 3: The structure of the agent. The user talks to the agent; the agent acts only through tools that pass a permission check; the same tools reach the digital world, the physical rig and other systems.

Why it matters. Digital first makes mistakes free and repeatable, and hidden truth stops us from grading our own homework. Counting cost means the agent must earn every measurement, as it would on a real roof. Building on an existing framework puts the effort into PV reasoning, not chat plumbing; permissions make the declared limits concrete, so a costly test waits for the user's yes. One set of tools for every world is what lets digital results carry over to the bench, and later to batteries and buildings.

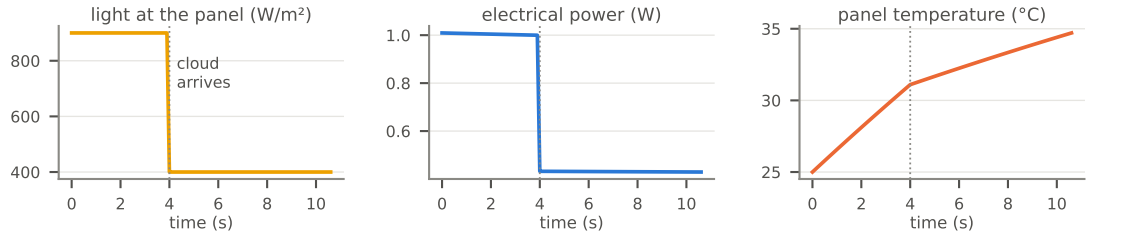
3 The preliminary tool: a simulator

Nobody starts from zero. A PV investigation simulator already runs on a laptop and is shared on GitHub with a README and two notebooks (Figure 4). It holds a 3D panel with a camera on a rail, an electrical and thermal model

with hidden settings, realistic sensors, a small set of permitted actions with budgets and limits, an evaluator that sees the hidden truth, and 875 automated tests. Table 1 maps it onto the loop of Figure 2.



(a) three frames from the simulated camera rail; the panel has eight cells and a small reference sensor beside it



(b) a hidden cloud at 4 s: light and power fall together, temperature only bends; the sensors report at 10 Hz with a 20 ms delay

Figure 4: What the simulator provides. (a) Pictures from the camera rail at three poses. (b) The three free channels during a hidden cloud: light and power fall together, temperature only changes its slope. Every reading carries its capture time and its delivery time.

Loop stage	What the tool provides today	Not yet
Sense	what the panel produces (voltage, current, power), how much light reaches it, how hot it is, where the camera points, and pictures on request; readings can arrive late, noisy or not at all, as in the field	shading that the camera can see
Believe	a short list of possible causes, each with a likelihood; every new reading raises the causes it fits and lowers the ones it contradicts; what is still open is written down with the evidence behind it	a cause that is on no list; likelihoods learned from data
Decide	a price for every action and a budget to spend; five hand-built controllers to compare against	planning by value of information; a language model
Act	look (move the camera, take a picture), test (change or probe the electrical operating point), wait, ask for help, stop safely; limits are checked before anything moves	battery or demand actions
Verify	every action reports whether it was done, with the reading that confirms it; predictions, from hand-written physics, are checked against what happened	a learned predictor (stretch goal)
Finish and interact	end as resolved, unresolved or help needed; ask one information source a question; replay any run in a browser	a live dialogue with the user (via OpenCode)

Table 1: The tool mapped onto the six boxes of the loop.

Why it matters. Everyone works against the same world, so results are comparable and repeatable. The hidden settings are what make fair scoring possible. The tool is also candid about its limits: in the current scene the cells are drawn in fixed colours, so a shaded cell looks clean to the camera. Making shading visible is the first step of the minimal goal, and it is the point at which the camera starts to earn its cost.

4 What the tool already shows: a preliminary example

This is a **preliminary example**: the worlds are simple and the agent loop is hand-built, not the agent of Figure 3. It shows the method, not final results. The second notebook, `multimodality_and_agent_loop.ipynb`, is this section in runnable form. Its five parts are the five steps below.

1. **Install and open.** No graphics card needed.

```
uv sync --locked --group dev
uv run --with jupyter --with matplotlib jupyter lab \
notebooks/multimodality_and_agent_loop.ipynb
```

   You see: the standard world loads and the notebook checks whether the camera works.

2. **Build a world with a hidden cause.** One line sets what is secretly wrong; the controller is never told.

```
make_config(dni_w_m2=300)    # a cloud
make_config(transmission=[1, 1, 0.3, 1, 1, 1, 1, 1])  # one shaded cell
make_config(series_resistance_ohm=1.0)   # a wiring fault
make_config(initial_temperature_c=60, ambient_temperature_c=40)  # a hot panel
```

   You see: four worlds that answer only through their sensors, as in Figure 4b.

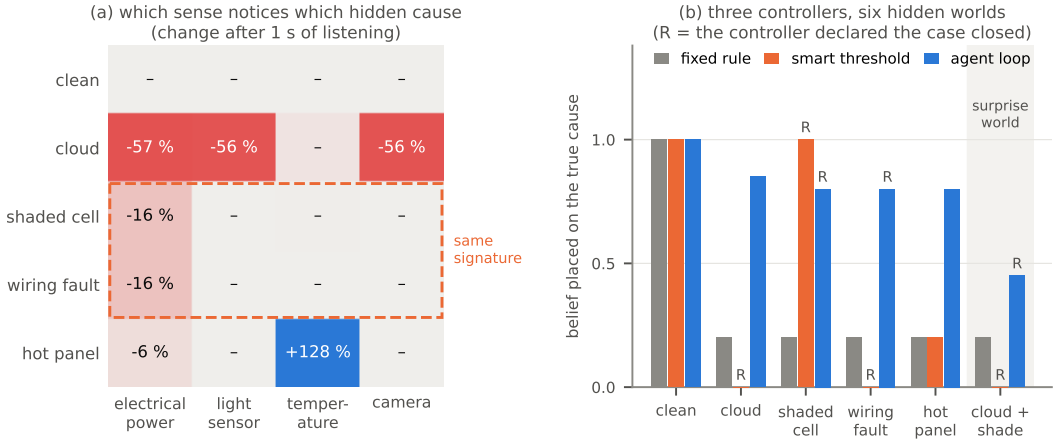


Figure 5: What the tutorial shows. (a) How much each sense changed, relative to a clean panel, after one second; a dash means less than 5 %. The dashed box marks the two causes no passive sense can separate. (b) Belief placed on the true cause by three controllers; "R" marks a controller that declared the case closed, so an R over an empty bar is a confident wrong answer.

3. **Look through every sense** (Parts 1 and 2). `passive_look(config)` takes one picture and listens for one second, then compares every sense with the clean panel. *You see:* Figure 5a. Power drops in every faulty world, so it is a good alarm and a useless diagnosis. The light sensor moves only under the cloud; the temperature only for the hot panel. Each added sense removes explanations, until two remain tied: the shaded cell and the wiring fault look identical from outside, however long you wait.
4. **Buy a test** (Part 3). `iv_sweep(config)` measures the current at six chosen voltages and divides by the clean panel's curve. *You see:* the wiring fault scales the whole curve by one factor (the ratio varies by 0.004 across the sweep); the shaded cell loses proportionally more at high voltage (the ratio varies by 0.10). The test costs production, so it is a decision, not a reflex.
5. **Run three controllers across six worlds** (Parts 4 and 5). `run(config, controller)` drives the loop; the controllers differ only in their believe and decide boxes. *You see:* Figure 5b. The fixed rule never commits. The smart threshold raises a false fault under the cloud, names the wrong fault for the wiring problem, and declares the case closed every time. The agent loop puts most of its belief on the true cause in every world and lists what it left untested. In the surprise world, a cloud and a shaded cell together, its printed trace reads: light down, image darker, power down by more than the light explains; one test bought; two causes named.

Why it matters. Five steps show that the value of a sense can be *measured*, as the number of explanations it removes; that sensing is a decision with a price; and that a confident wrong answer can be *caught and scored*. Those are the three things the research question rests on, and anyone can reproduce all three on a laptop in an afternoon, before any hardware is built.

5 What can be developed next

Figure 6 is the roadmap; both goal columns are **illustrative**, not a fixed list. The **illustrative minimal goal** makes shading visible, builds the agent of Figure 3 and connects the same tools to a bench rig for a live demonstration. The **illustrative stretch goals** add a predictor that learns from the agent's own investigations, causes on no list, goals given in words, planning by value of information, storage and demand, and the comparison with a general language-model agent that defines the contribution. Whether the predictor comes from physics, data or both is part of that work.

	Have now (Oct 2026)	Illustrative minimal goal	Illustrative stretch goals
Simulator and predictor	simulated 8-cell panel and camera rail; hidden cloud, shading, wiring fault and overheating; hand-written physics predicts each test	shading the camera can see; dust, objects, moving shadows; a simple building: tariff, weather, occupancy	a predictor learned from the agent's own tests: what happens next, what acting or waiting costs; real-module, battery and demand models
Agent	three hand-built controllers; the agent loop beats a smart threshold and can name two causes at once	an agent in OpenCode that reads images, uses the models as tools and reports back to the user	an "unlisted cause" answer; goals given in words; planning by value of information; comparison with LLMs
Proof and hardware	evaluator with hidden truth, paired scoring and replay viewer; two teaching notebooks and a README	a bench rig: small panel, lamp, controllable load, camera on a rail; a live demonstration	held-out cases with causes on no list (Case A); compound environment and demand scenarios (Case B)

Figure 6: Roadmap. Each cell is testable against the one to its left, with the same evaluator and the same hidden cases.

Why it matters. Each step is measurable against the last. The minimal goal is a result in its own right; the stretch goals turn it into evidence, and the predictor moves the agent from explaining what went wrong to anticipating what will happen.